

게이지 + 대시/텔레포트 계열 사전 준비 가이드

목표는 스킬을 미리 맞혀서 코드를 도박하는 것이 아니다. **예측이 맞으면 빠르게 연결하고, 틀려도 같은 기록을 새 스킬 적용 순서로 재사용하는 것이다.**

현재 가설

게이지가 존재하고, 게이지를 소비해 평소보다 더 빨리/멀리 공에 접근하는 이동/접촉 강화 스킬이 생긴다. 대시, 공중 대시, 슈퍼점프, 제한적 텔레포트가 같은 큰 범주다.

확정된 사실이 아님

게이지의 범위, 충전법, 사용 비용, action 형식, 정확한 이동량, 발동 순서는 전부 공개 당일 확인해야 한다.

왜 이동을 우선 대비하나

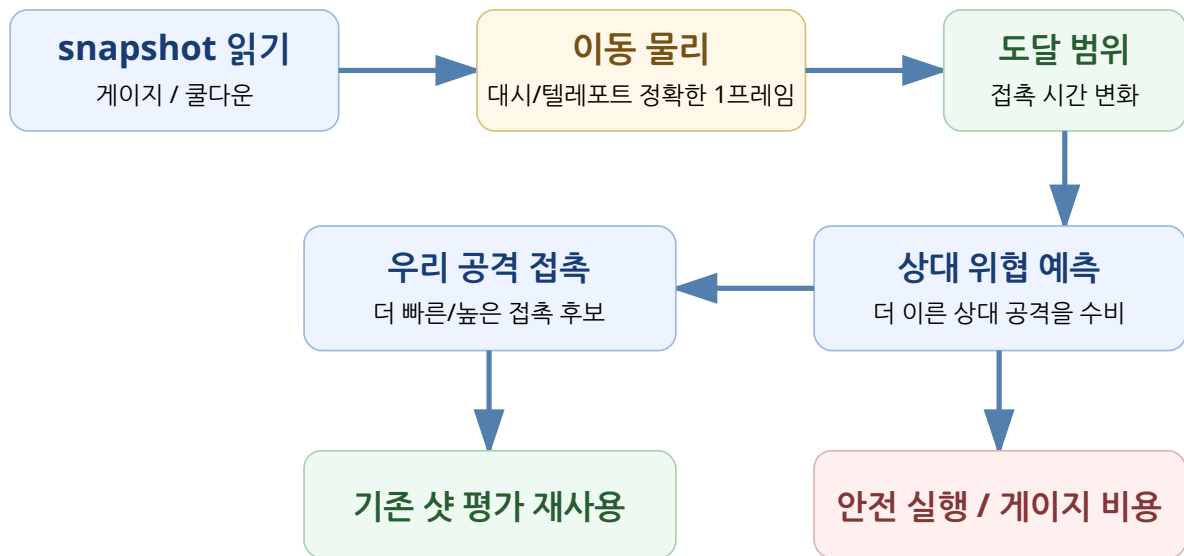
1. 기존 소스에 **skill/gauge.js** 흔적이 있어 자원 시스템 가능성을 높게 본다.
2. 운영진은 순수 수비형이 불리할 수 있다고 했다.
3. 이동기는 "공이 얼마나 빠른가"가 아니라 **누가 언제 공에 닿을 수 있는가**를 바꾼다.

가장 위험한 변화

상대 예상 접촉 8프레임 뒤 -> 스킬 사용 후 3프레임 뒤

수비 위치를 잘 골라도 준비 시간이 5프레임 사라지면 기존 계산 자체가 무너질 수 있다.

1. v2.1에서 어디가 영향을 받나



중요

현재 `SKILL.reachActions()`는 이름은 준비되어 있지만 v2.1 core에서 실제로 호출되지 않는다. 대시가 공개되어도 이 함수만 채우면 아무 변화가 없다. 실제로는 `canPlayerReachBallAt()` / `interceptTrajectory()`와 우리 접촉 planner에 연결해야 한다.

2. 현장에서 고치는 순서

0) 먼저 실측

게이지 이름과 범위, 발동 input, 지상/공중 사용 가능, 이동 거리, 지속 프레임, collision 가능 여부, held input에서 반복 발동 여부, 프레임 처리 순서를 먼저 확인한다.

1) `SKILL.read()`

게이지와 준비 상태를 읽는다. 이 단계에서는 행동을 바꾸지 않는다.

```
{ selfGauge, oppGauge, selfReady, oppReady, cooldown, active }
```

2) action 형식

새 `skill` 필드가 생기면 `extraActions()`만 수정해서는 안 된다. `action()`, `cloneAction()`, `uncanonicalize()`, `executeSafely()`, `decide()` 반환까지 함께 확장한다.

3) 정확한 이동 물리

전략보다 먼저 한 프레임의 이동을 브라우저와 맞춘다. 대시 발동 프레임에 즉시 이동하는지, 다음 프레임부터 이동하는지가 접촉 시간에 직접 영향을 준다.

4) 상대 skill reach를 먼저 반영

상대 게이지가 충분하면 수비기는 상대가 스킬을 사용한다고 가정하고 가장 이른 접촉을 계산한다.

5) 우리 공격 skill contact 추가

일반 접촉보다 더 이르거나 높은 접촉이 생기면 그 접촉에서 기존 power shot 후보를 평가한다.

6) 마지막에 게이지 비용 최적화

물리와 접촉 시간이 맞은 뒤에야 "아낄까 쓸까"를 튜닝한다.

3. 대시와 텔레포트는 어떻게 다르게 처리하나

	대시	텔레포트
핵심	몇 프레임에 걸친 빠른 이동	위치가 한 번에 크게 바뀜
필수 실측	속도, 지속시간, 조향 가능 여부	목표 선택 방식, 즉시 collision 여부
Planner	발동 프레임 + 방향 후보를 작은 수로 열거	가능한 도착점/방향 후보를 열거
위험	held input 중복 사용	경계/네트/충돌 순서 오판

공통 전략

스킬 그 자체의 점수를 따로 만들기보다, 스킬로 만든 새로운 접촉에서 기존 공격 평가를 다시 돌린다.

skill 가치 = skill 접촉 후 최선 공격 - normal 접촉 후 최선 공격 - 게이지 비용 - 실패 위험

4. 게이지 사용 정책 - 처음에는 단순하게

공격

- 스킬로 접촉이 눈에 띄게 빨라질 때
- 더 이른 접촉이 실제로 더 좋은 공격으로 이어질 때
- 자기 코트 실점 위험이 없을 때
- 스킬 없이도 확정 득점이면 낭비하지 않기

수비

- 스킬 없이는 거의 확정 실점
- 스킬이면 살릴 수 있음

이런 처음에는 게이지를 쓰는 쪽으로 강하게 허용하는 것이 안전하다. 자원 보존 가치는 실제 게이지 유지 규칙을 본 뒤 조정한다.

운영진 힌트와 연결

게이지가 시간이 지나며 차는 구조라면 긴 랠리 자체가 상대방에게 필살기 기회를 줄 수 있다. 이 경우 "수비만 오래 버티기"가 구조적으로 불리해질 수 있다.

5. 반드시 남길 로그

```
selfGauge / oppGauge
normalEarliestContact / skillEarliestContact
skillContactGain
selectedSkill / activationFrame
predicted player x,y at contact
predicted ball x,y at contact
gauge before/after
actual contact frame
score before/after
```

6. 예측이 틀렸을 때

실제 스킬	이 기록에서 재사용할 부분
강화 스매시	snapshot/action 연결 -> 정확한 공 속도 변화 -> 기존 `evaluateOutgoingTrajectory()`에 새 후보 추가
슈퍼점프	대시와 동일하게 "새로운 접촉 가능성" 문제로 처리
히트박스 확대	`canPlayerReachBallAt()`의 충돌 범위와 우리 접촉 planner 수정
공 중력/궤적 변화	`SKILL.ballFrameHook()`과 정확한 공 시뮬레이션부터 수정
방어 스킬	상대 intercept 평가와 우리 emergency save 후보에 연결

즉, 가설이 빗나가도 순서는 같다.

무슨 값이 새로 생겼나 -> 엔진 한 프레임에서 무엇이 달라졌나 -> 접촉 가능성이 어떻게 달라졌나 -> 기존 평가기를 어디에 재사용하나 -> 마지막에 자원 사용 정책을 튜닝한다.

7. 현장 최우선 체크리스트

1. 공개 문서와 소스 원문 확보
2. 새 snapshot/action 예시 확보
3. 10개 짧은 프레임 실측
4. 스킬 OFF 회귀 100% 동일 확인
5. 상대 skill earliest contact를 먼저 수비에 연결
6. 우리 공격 candidate 연결
7. 좌우/네트 경계 테스트
8. 두 기존 상대 코드와 동일 seed A/B

하지 말 것

정확한 발동 순서가 불명확한 상태에서 대충 displacement를 넣고 전략 튜닝부터 시작하지 않는다.
v2.1이 강한 이유는 "실제 접촉과 계산 접촉을 최대한 맞춘 것"이기 때문이다.